

Global Multi-Probing for Maximum Inner Product Search with Adaptive Early Termination

Bolong Zheng¹, Xi Zhao¹, Kai Zheng², Nguyen Quoc Viet Hung³,
Xiaofang Zhou⁴, Christian S. Jensen⁵

¹Huazhong University of Science and Technology, ²University of Electronic Science and Technology of China,
³Griffith University, ⁴Hong Kong University of Science and Technology, ⁵Aalborg University
bolongzheng@hust.edu.cn, zhaoxi@hust.edu.cn, zhengkai@uestc.edu.cn
quocviet.hung.nguyen@griffith.edu.au, zxf@cse.ust.hk, csj@cs.aau.dk

ABSTRACT

Maximum inner product search (MIPS) in high-dimensional spaces has wide applications but is computationally expensive due to the curse of dimensionality. Existing studies employ asymmetric transformations that reduce the MIPS problem to a nearest neighbor search (NNS) problem, which can be solved using locality-sensitive hashing (LSH). However, the asymmetric transformations introduce either distortion errors or data distribution imbalances. In addition, LSH is applied without taking into account the properties of the inner product. In this paper, we propose a global multi-probing (GMP) algorithm for MIPS on large-scale, high-dimensional data. First, we design a novel transformation, called random XBOX transformation (RXT), that eliminates distortion errors and reduces data distribution imbalances. Second, we develop a global multi-probing strategy as well as an adaptive early termination condition that finds results quickly and offers theoretical guarantees. Extensive experiments with real-world data offer evidence that GMP is capable of outperforming existing proposals in terms of both accuracy and efficiency.

PVLDB Reference Format:

Bolong Zheng¹, Xi Zhao¹, Kai Zheng², Nguyen Quoc Viet Hung³,
Xiaofang Zhou⁴, Christian S. Jensen⁵. Global Multi-Probing for Maximum
Inner Product Search with Adaptive Early Termination. PVLDB, 14(1):
XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at
http://vldb.org/pvldb/format_vol14.html.

1 INTRODUCTION

Given a query point q and a dataset $\mathcal{D}$, the maximum inner product search (MIPS) problem aims to find a point x in $\mathcal{D}$ that has the maximum inner product $q^\top x$. The MIPS problem in high-dimensional spaces is core functionality in a wide variety of applications, such as matrix factorization based recommendation [1, 29], multi-class label prediction [10, 19], similar-item retrieval [37], structural SVM [20], and deep learning [15], to name but a few.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

A basic solution to the MIPS problem is the space partitioning tree-based method [21, 22, 30], which works well in low dimensional spaces. However, when the dimensionality increases to several hundred or higher, the search time increases exponentially. This phenomenon is called the “curse of dimensionality.” Since the MIPS problem is as complex as nearest neighbor search (NNS) [2, 5, 7], it is natural to consider the inner product as a similarity metric and apply directly locality-sensitive hashing (LSH) [3, 23, 24, 35, 40, 41]. However, to work correctly, LSH requires that the metric satisfies the condition that for any point q , the point that has the largest similarity to q is q itself. Unfortunately, the inner product does not satisfy this condition. That is, a point x always exists that is different from q and such that $q^\top x > q^\top q$.

Existing studies adopt asymmetric transformations that convert the MIPS problem into NNS, and then use LSH to solve the NNS problem. Three main asymmetric transformation methods exist: (1) the L2-Transformation [32], (2) the Correlation-Transformation [33], and (3) the XBOX-Transformation [4, 38]. However, each of these transformations suffers from shortcomings that impact their performance negatively. Thus, the L2-Transformation and Correlation-Transformation introduce distortion errors. The XBOX-Transformation avoids distortion errors, but brings data distribution imbalances to the transformed dataset. Having performed a transformation, these studies apply LSH to examine the points without considering the properties of the inner product. Most studies use the basic (K, L) -bucketing strategy. H2-ALSH [17] improves the query efficiency via QALSH [16], but it still disregards the data distribution imbalance brought by the XBOX-Transformation.

We propose a global multi-probing algorithm (GMP) for computing the MIPS problem on large-scale, high-dimensional datasets. In GMP, we develop a novel asymmetric transformation, called random XBOX-Transformation (RXT), that maps the points in two different directions randomly so that the points are distributed more evenly. To process a query, we use an accurate and efficient global multi-probing strategy to examine candidate points. To improve the quality of candidates, we exploit the strong relationship between the inner product and the so-called quantization distance between the query point and a candidate point. In addition, we develop an adaptive early termination condition that enables a better trade-off between accuracy and efficiency.

The major contributions are summarized as follows:

- We develop a novel asymmetric transformation method RXT that reduces data distribution imbalance.

Table 1: Summary of Notations

Notation	Definition
$\mathcal{D}$	Dataset of points in $\mathbb{R}^d$
$n = \mathcal{D} $	Dataset cardinality
d	Dimensionality
x	A point in $\mathcal{D}$
$h(x)$	Hash functions

- We propose an efficient and accurate method called GMP to answer MIPS queries that exploits the properties of the inner product to examine high quality candidates. In addition, we develop an adaptive early termination that determines its termination conditions adaptively.
- We conduct an extensive performance study using real datasets that covers the state-of-the-art methods for MIPS, finding that GMP is efficient as well as accurate in terms of both the overall ratio and recall.

The rest of the paper is organized as follows. Section 2 presents the problem setting and several key algorithms used in existing LSH methods for MIPS. Section 3 introduces the background knowledge. Section 4 explains the construction and query algorithms of the GMP, and Section 5 establishes its performance guarantees. Section 6 covers experimental studies that offer insight into the performance of the proposed GMP and the main competitors. Section 7 reviews related work. Finally, Section 8 concludes the paper.

2 PRELIMINARIES

We present the problem definition and basic idea of MIPS. Frequently used notation is summarized in Table 1.

2.1 Problem Definition

DEFINITION 1 (MAXIMUM INNER PRODUCT SEARCH). Given a dataset $\mathcal{D} \subset \mathbb{R}^d$ that contains n points (also called vectors) and a query point $q \in \mathbb{R}^d$, the maximum inner product search (MIPS) returns a point p in $\mathcal{D}$ that has the maximum inner product with q ,

$$p = \arg \max_{x \in \mathcal{D}} q^\top x \quad (1)$$

The MIPS problem is related to the exact nearest neighbor search (NNS) problem, where we aim to find a point x that has the smallest distance, or largest similarity, to q .

- For Cosine similarity $\text{sim}(x, y) = \frac{x^\top y}{\|x\| \|y\|}$, we have:

$$p = \arg \max_{x \in \mathcal{D}} \frac{q^\top x}{\|q\| \|x\|} = \arg \max_{x \in \mathcal{D}} \frac{q^\top x}{\|x\|} \quad (2)$$

- For Euclidean distance $\text{dist}(x, y) = \|x - y\|$, we have:

$$p = \arg \min_{x \in \mathcal{D}} \|q, x\| = \arg \min_{x \in \mathcal{D}} (\|x\| - 2q^\top x). \quad (3)$$

As the exact NNS problem is inherently computational expensive in high-dimensional space due to the ‘‘curse of dimensionality,’’ it is attractive and generally sufficient to find an approximate nearest neighbor.

DEFINITION 2 (c-APPROXIMATE NEAREST NEIGHBOR SEARCH). Assume a query point q , and an approximation ratio $0 < c < 1$.

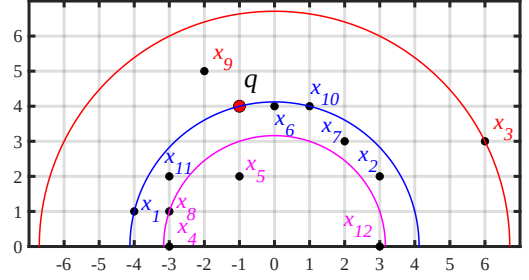


Figure 1: A Running Example

Let $\text{sim}(\cdot)$ be the similarity function, and let x^* be the exact nearest neighbor of q in $\mathcal{D}$. A c -approximate nearest neighbor search (c -ANNS) returns a point $x \in \mathcal{D}$ such that $\text{sim}(x, q) \geq c \cdot \text{sim}(x^*, q)$.

In this paper, we focus on the transformation that converts a c -maximum inner product search (c -MIPS) to a c -approximate nearest neighbor search (c -ANNS). We define formally the c -MIPS as follows.

DEFINITION 3 (c-MAXIMUM INNER PRODUCT SEARCH). Assume a query point q , and an approximation ratio $0 < c < 1$. Let x^* be the point that has the maximum inner product with q . A c -maximum inner product search (c -MIPS) returns a point $x \in \mathcal{D}$ such that $q^\top x \geq c \cdot q^\top x^*$.

In the following, we use MIPS and c -MIPS interchangeably when this does not cause ambiguity.

EXAMPLE 1. Fig. 1 shows a query point q and a dataset with 12 points. x_9 is the exact MIPS of q with $q^\top x_9 = 22$. When $c = 0.5$, the c -MIPS returns any point in $\{x_6, x_9, x_{10}, x_{11}\}$ with inner product larger than $0.5 \times 22 = 11$.

2.2 Locality Sensitive Hashing

Locality Sensitive Hashing (LSH) is a family of hash functions that enables c -ANNS in sublinear time with constant probability. An important property of LSH is that the points with larger similarity have higher collision probability on their hash values.

DEFINITION 4 (LOCALITY SENSITIVE HASHING). Given a similarity S_0 , an approximation ratio $0 < c < 1$, probability values p_1 and p_2 , where $p_1 > p_2$, a family $\mathcal{H} = \{h : \mathbb{R}^d \rightarrow U\}$ is called (S_0, cS_0, p_1, p_2) -locality sensitive, if for any $x, y \in \mathbb{R}^d$, it satisfies the following two conditions:

- (1) If $\text{sim}(x, y) \geq S_0$ then $\Pr[h(x) = h(y)] \geq p_1$
- (2) If $\text{sim}(x, y) \leq cS_0$ then $\Pr[h(x) = h(y)] \leq p_2$

LSH is a generic framework and has different hash functions for each similarity metric. One study [6] proposes an LSH family, called signed random projection (SRP), for Cosine similarity, which is defined as follows.

$$h(x) = \text{sign}(a^\top x), \quad (4)$$

where a is a random vector with each dimension $a_i \sim N(0, 1)$. Another study [9] proposes an LSH family, called fixed window w random projection (w -RP), for Euclidean distance, which is defined

as follows.

$$h(x) = \lfloor \frac{a^\top x + b}{w} \rfloor, \quad (5)$$

where a is a random vector with each dimension $a_i \sim N(0, 1)$, b is a real number uniformly drawn from $[0, w)$, and w is a user-specified window size.

3 FROM MIPS TO NNS

Given that the definitions of c -MIPS and c -ANNS are similar, the idea of considering the inner product as a similarity metric and then applying LSH to solve MIPS is nature. However, a study [32] shows that the classical LSH framework is restrictive when solving MIPS. Therefore, existing proposals adopt an asymmetric transformation that maps the data points from the original space to a transformed space, and return the c -ANNS results in the transformed space.

3.1 Asymmetric Transformation

Assume that we adopt a symmetric transformation $P(\cdot)$ for both the data points and the query point q and query q equals to a point x . The transformations of q and x are identical, so $\text{sim}(P(q), P(x)) = 1$. Therefore, x will always be returned as the NN of q in the transformed space— x is the point with the largest similarity to q . However, this contradicts the fact that x is not necessarily the point with the maximum inner product with q . Therefore, an asymmetric transformation that preserves the relation between the inner product in the original space and the similarity in the transformed space is required.

To enable a better understanding, we introduce the existing asymmetric transformation methods, each of which consists of a function $P(x)$ for data point x and a function $Q(q)$ for query point q . Before the transformation, we scale down the norm of each point such that $\|x\| \leq U < 1$ for some $U < 1$.

3.1.1 L2-Transformation [32]. The L2-Transformation adopts two functions $P : \mathcal{R}^d \mapsto \mathcal{R}^{d+m}$ and $Q : \mathcal{R}^d \mapsto \mathcal{R}^{d+m}$ that are defined as follows.

$$P(x) = [x; \|x\|^2; \|x\|^4; \dots; \|x\|^{2m}]$$

$$Q(q) = [q; 1/2; 1/2; \dots; 1/2],$$

where m is a positive integer. By computing the Euclidean distance between $P(x)$ and $Q(q)$, we have:

$$\text{dist}(P(x), Q(q)) = \|P(x), Q(q)\|^2 = (1 + m/4) - 2q^\top x + \|x\|^{2m+1}$$

As $\|x\| \leq U < 1$, $\|x\|^{2m+1} \rightarrow 0$ when m is sufficiently large. Therefore, we have:

$$\text{dist}(P(x), Q(q)) \simeq (1 + m/4) - 2q^\top x \quad (6)$$

In Eq. 6, MIPS is transformed to NNS under the Euclidean space.

3.1.2 Correlation-Transformation [33]. Two functions $P : \mathcal{R}^d \mapsto \mathcal{R}^{d+m}$ and $Q : \mathcal{R}^d \mapsto \mathcal{R}^{d+m}$ are adopted that are defined as follows.

$$P(x) = [x; 1/2 - \|x\|^2; 1/2 - \|x\|^4; \dots; 1/2 - \|x\|^{2m}]$$

$$Q(q) = [q; 0; 0; \dots; 0].$$

The Cosine similarity is computed as follows.

$$\text{sim}(P(x), Q(q)) = \frac{Q(q)^\top P(x)}{\|Q(q)\| \|P(x)\|} = \frac{q^\top x}{\sqrt{m/4 + \|x\|^{2m+1}}}.$$

Likewise, as $\|x\|^{2m+1} \rightarrow 0$, we have:

$$\text{sim}(P(x), Q(q)) \simeq \frac{q^\top x}{\sqrt{m/4}} \quad (7)$$

In Eq. 7, MIPS is transformed to NNS in Cosine space.

3.1.3 XBOX-Transformation [17, 21, 28, 38]. Let M denote the maximum norm of points in $\mathcal{D}$, i.e., $M = \max_x \|x\|$. Similarly, the XBOX-Transformation adopts two functions $P : \mathcal{R}^d \mapsto \mathcal{R}^{d+1}$ and $Q : \mathcal{R}^d \mapsto \mathcal{R}^{d+1}$, defined as follows.

$$P(x) = [x; \sqrt{M^2 - \|x\|^2}] \quad (8)$$

$$Q(q) = [q; 0] \quad (9)$$

By computing the Cosine similarity and Euclidean distance between $P(x)$ and $Q(q)$, we have:

$$\text{sim}(P(x), Q(q)) = \frac{q^\top x}{\|q\| \cdot M} \quad (10)$$

$$\text{dist}(P(x), Q(q)) = \|P(x), Q(q)\|^2 = \|q\|^2 + M^2 - 2q^\top x \quad (11)$$

Unlike the L2-Transformation and the Correlation-Transformation, the XBOX-Transformation is an exact transformation, since both the Cosine similarity and Euclidean distance between $P(x)$ and $Q(q)$ can be computed as the inner product between x and q without distortion. So we propose a variant of the XBOX-Transformation as our transformation method.

3.2 Cosine Similarity vs Euclidean Distance

After the transformation, a c -MIPS query is converted to a c' -ANNS query that can be solved directly using existing LSH methods. Two LSH families are adopted widely: (1) the SRP in Eq. 4 for Cosine similarity and (2) the w -RP in Eq. 5 for Euclidean distance. Note that $c \neq c'$ and the approximation ratio c' is a key factor that determines the difficulty of answering the c' -ANNS query, which had complexity $O(n^{1/c'})$ using the (K, L) -bucketing strategy [12, 13, 18]. Moreover, c' differs for similarity measures. Given $c' = c_1$ for Cosine similarity and $c' = c_2$ for Euclidean distance, we demonstrate that c_1 always exceeds c_2 as follows.

DEFINITION 5 (POINT ANGLE). For two data points x and y , the angle θ between the vectors of data points x and y is defined as follows.

$$\theta_{x,y} = \arccos \frac{x^\top y}{\|x\| \cdot \|y\|}. \quad (12)$$

Assume that $\|q\| = M$. Let x^* be q 's exact MIPS point, so $P(x^*)$ is $Q(q)$'s exact NNS point in the transformed space. Since $q^\top x^* < M^2$, we have $q^\top x^* = l \cdot M^2$, $0 < l < 1$. The c -MIPS query returns a point x where $P(x)$ is the result of the c' -ANNS query, so we have $q^\top x = c \cdot l \cdot M^2$. According to Eq. 8, we know that $\|P(x)\| = \|P(x^*)\| = M$.

For Cosine similarity, we have:

$$c_1 = \frac{\theta_{P(x), Q(q)}}{\theta_{P(x^*), Q(q)}} = \frac{\arccos \frac{q^\top x}{M \cdot M}}{\arccos \frac{q^\top x^*}{M \cdot M}} = \frac{\arccos(c \cdot l)}{\arccos(l)}.$$

For Euclidean distance, we have:

$$c_2 = \frac{\|P(x), Q(q)\|}{\|P(x^*), Q(q)\|} = \frac{\sqrt{2M^2 - 2c \cdot l \cdot M^2}}{\sqrt{2M^2 - 2 \cdot l \cdot M^2}} = \frac{\sqrt{1 - c \cdot l}}{\sqrt{1 - l}}.$$

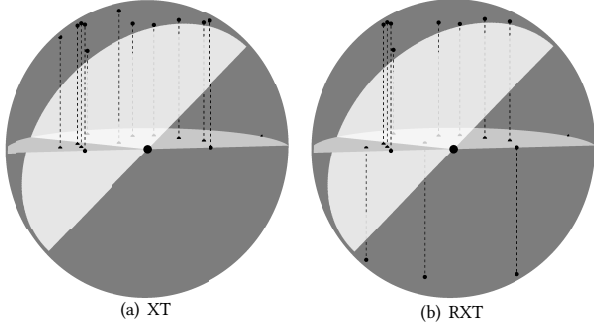


Figure 2: Comparison of RXT with XT

It holds that $c_1 > c_2$ for any $c, l \in (0, 1)$. Therefore, we choose the Cosine similarity as the metric and SRP as the LSH family for the c -ANNS in the transformed space.

4 OUR SOLUTION

We proceed to introduce our solution, GMP, for computing c -MIPS queries. For the data preprocessing, we adopt a norm ranging strategy [17, 38] that divides the data points into disjoint partitions based on the norm, and we develop a random XBOX-Transformation (RXT) that maps the data points into a transformed space. For index construction, we employ SRP as the LSH family and utilize the (K, L) -bucketing strategy that builds hash tables for each partition. To examine candidate points, we use a global multi-probe strategy to generate a probing sequence. In addition, we develop an adaptive condition to check if the process can terminate early.

4.1 Data Preprocessing

4.1.1 Data Partitioning. Intuitively, a large norm $\|x\|$ leads to a large inner product $q^\top x$ with a high probability, since $q^\top x$ is directly proportional to the norm $\|x\|$. Existing studies use so-called norm ranging to divide the data points into disjoint partitions, $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s$, according to their norms. We follow a strategy similar to that of an existing study [17] that uses two parameters b_0 and N_0 to bound the range of norms, where b is the norm ratio and N_0 is the maximum number of points in each partition. The partitions satisfy three conditions:

- (1) For any two points $x \in \mathcal{D}_i$ and $y \in \mathcal{D}_j$, if $i < j$, we have $\|x\| \leq \|y\|$.
- (2) For any two points $x, y \in \mathcal{D}_i$, if $\|x\| \leq \|y\|$, we have $b_0 \cdot \|y\| < \|x\|$, $b_0 \in (0, 1)$.
- (3) For each partition $\mathcal{D}_i$, we have $|\mathcal{D}_i| < N_0$.

EXAMPLE 2. Assuming $N_0 = 6$ and $b = 0.7$, the dataset in Fig. 1 is divided into three partitions, as indicated by the three different colors.

4.1.2 Random XBOX-Transformation. We develop a novel random XBOX-Transformation (RXT) that converts a c -MIPS query to a c' -ANNS query. Like the XBOX-Transformation (XT), RXT has two functions $P : \mathcal{R}^d \mapsto \mathcal{R}^{d+1}$ and $Q : \mathcal{R}^d \mapsto \mathcal{R}^{d+1}$, which are

Table 2: Comparison of Transformation Methods

Algorithm	Trans.	Distort.	Trans. Space
L2-ALSH [32]	L2	Yes	Euclidean
Sign-ALSH [33]	Correlation	Yes	Cosine
H2-ALSH [17]	XBOX	No	Euclidean
Simple-LSH [28]			Cosine
Range-LSH [38]			
RPT [21]			
GMP	RXT	No	Cosine

defined as follows.

$$\begin{aligned} P(x) &= [x; r \cdot \sqrt{M^2 - \|x\|^2}] \\ Q(q) &= [q; 0], \end{aligned} \quad (13)$$

where $r \in \{1, -1\}$ is a random variable and $\Pr[r = 1] = \Pr[r = -1] = \frac{1}{2}$. Note that RXT still satisfies Eq. 10. The difference of RXT from XT is that the sign of the $(d + 1)$ -st dimension of each transformed point $P(x)$ has 50% probability to be -1 . Fig. 2 shows a comparison between RXT and XT.

After the transformation using RXT, we adopt SRP that maps all $P(x)$ in the transformed space into a projected space. For each hash function $h(\cdot)$, we randomly choose a vector a so that $h(P(x)) = \text{sign}(a^\top P(x))$. For simplicity, we denote $h(P(x))$ by $h(x)$. Then we use the (K, L) -bucketing strategy to build hash tables in the projected space. We observe that the point distributions in hash buckets are more balanced when using RXT than when using XT. We analyze the advantage of RXT by considering the probabilities of $h(x) = 1$ and $h(x) = -1$.

LEMMA 1. Using XT, the number of points with hash value 1 is always different with the number of points with hash value -1 ; while using RXT, it is likely that the number of points with hash value 1 equals to the number of points with hash value -1 .

PROOF. For XT, we have $h(x) = \text{sign}(a^\top P(x)) = \text{sign}(\sum_{i=1}^d x_i a_i + \sqrt{M^2 - \|x\|^2} \cdot a_{d+1})$. Let $A = \frac{\sum_{i=1}^d x_i a_i}{\|x\|}$, we have $A \sim N(0, 1)$ according to the property of the sum of the normal distribution since $a_i \sim N(0, 1)$. Thus, $h(x) = \text{sign}(\|x\|A + \sqrt{M^2 - \|x\|^2} \cdot a_{d+1})$.

$$\begin{aligned} \Pr[h(x) = -1] &= \Pr[A < -\frac{\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|}] \\ &= \phi(-\frac{\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|}), \end{aligned}$$

where $\phi(\cdot)$ is the cumulative distribution function of the standard normal distribution. Therefore, if $a_{d+1} > 0$, then $-\frac{\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|} < 0$ for any a point x . In this case, it is likely that more than $n/2$ points have hash values 1. Otherwise, it is likely that more than $n/2$ points have hash values -1 .

For RXT, since $\Pr[r = 1] = \Pr[r = -1] = 1/2$, we have

$$\begin{aligned} \Pr[h(x) = -1] &= \phi(-\frac{r\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|}) \\ &= \frac{1}{2} \left[\phi(-\frac{\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|}) + \phi(\frac{\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|}) \right] = \frac{1}{2}. \end{aligned}$$

No matter what a_{d+1} is, it is expected that $n/2$ points have hash values 1, which completes the proof. $\square$

A hash function $h(\cdot)$ can be considered as a hyperplane that partitions the projected space into two sides. Lemma 1 implies that the points are more likely to be mapped to one side of the hyperplane using XT; and that they are likely to be evenly mapped to two sides of the hyperplane using RXT. Therefore, RXT reduces the data distribution imbalance after the transformation, which is beneficial for the subsequent point probing process. Table 2 shows a comparison between RXT and existing transformations.

4.2 Building Hash Tables

As mentioned, for each data point $P(x)$ in the transformed space, we adopt SRP as the LSH family so that $h(x) = \text{sign}(a^\top P(x))$. According to the property of SRP, we have the following:

LEMMA 2. For a query point q and a data point x , the angle between $Q(q)$ and $P(x)$ in the transformed space is:

$$\theta = \arccos \frac{Q(q)^\top P(x)}{\|Q(q)\| \cdot \|P(x)\|} = \arccos \frac{q^\top x}{\|q\| \cdot M}.$$

Without loss of generality, we assume that $\|q\| = 1$. Since M is a constant, the inner product of x and q is inversely proportional to the angle between $Q(q)$ and $P(x)$. The colliding probability of $h(x)$ and $h(q)$ is computed as follows [28].

$$\Pr[h(x) = h(q)] = 1 - \frac{\theta}{\pi}. \quad (14)$$

Next, we build hash tables using the (K, L) -bucketing strategy. After the norm ranging, for each partition $\mathcal{D}_i$, we build L hash tables and choose K hash functions $\mathcal{H} = \{h_1, \dots, h_K\}$ for each hash table. This yields $K \cdot L$ hash functions and 2^{KL} hash buckets for each partition. To simplify the computation and reduce the number of hash functions, all s partitions use the same $K \cdot L$ hash functions to build the index.

4.3 Global Multi-Probing

We propose a global multi-probing (GMP) algorithm that finds the point with the largest Cosine similarity to $Q(q)$ in the transformed space. GMP is inspired to some extent by Multi-Probe [26]. Multi-Probe derives a local probing sequence for each hash table to check the candidate points; and examines hash tables sequentially. In contrast, GMP generates a global probing sequence from all L hash tables by using a so-called quantization distance (QD) [24] that evaluates the probability that a hash bucket contains the nearest neighbors, so that we do not have to access the hash tables one by one. In addition, GMP develops an adaptive early terminate (AET) condition that reduces the number of points to be examined.

4.3.1 Overview of the Query Phase. In the query phase, we first compute $Q(q)$ based on Eq. 13 and project $Q(q)$ with the K hash functions for each hash table. Since the dataset is divided into s partitions, we examine them in descending order of the maximum norm. To generate a global probing sequence, we evaluate the probability that a hash bucket B contains the nearest neighbour via a so-called quantization distance (QD). The smaller the QD is, the higher the probability that B contains the nearest neighbour is. For each partition $\mathcal{D}_i$, we determine which bucket to probe next

among all $L \cdot 2^K$ buckets based on the global probing sequence. This process terminates when a termination condition is satisfied.

We consider three problems in GMP:

- (1) Why is QD an effective indicator for probing hash buckets in all hash tables?
- (2) How to efficiently find the next hash bucket to probe among all unseen hash buckets?
- (3) How to set a proper termination condition to balance the accuracy and the efficiency?

4.3.2 Quantization Distance. Let $\zeta(x)$ be the projected vector of $P(x)$ under K hash functions, where $h_i(x) = \text{sign}(\zeta_i(x))$ and $\zeta_i(x) = a_i^\top P(x)$, $1 \leq i \leq K$. The quantization distance is defined as follows.

DEFINITION 6 (QUANTIZATION DISTANCE). For a query point q and a bucket $B = (b_1, b_2, \dots, b_K)$, the quantization distance between them is computed as follows.

$$QD(q, B) = \sum_{h_i(q) \neq b_i} |\zeta_i(q)|^2, 1 \leq i \leq K. \quad (15)$$

$QD(q, B)$ is a random variable since it is the sum of a set of random variables $\zeta_i(q)$. Let $B(x)$ be the hash bucket of a data point x , we use the cumulative distribution function $\Pr[QD(q, B(x)) \leq \omega]$ to describe the distribution of $QD(q, B(x))$. Note that, the QD between q and $B(x)$ is computed by using the same K hash functions in the same hash table. To show that QD is legal for ranking buckets from different hash tables, we first prove that $\Pr[QD(q, B(x)) \leq \omega]$ is only dependent on q and x .

LEMMA 3. Given a data point x with the angle $\theta_{P(x), Q(q)} = \theta$ between q and x in the transformed space, the cumulative distribution function of $QD(q, B(x))$ is determined only by θ and is independent on the hash table of $B(x)$.

For simplicity, we denote the cdf of QD as follows.

$$\varphi(\omega; \theta) = \Pr[QD(q, B(x)) \leq \omega] \quad (16)$$

LEMMA 4. $\varphi(\omega; \theta)$ monotonically decreases with θ . In other words, for $0 \leq \theta_1 < \theta_2 \leq \pi$ and $\omega > 0$, we have $\varphi(\omega; \theta_1) > \varphi(\omega; \theta_2)$.

It indicates that the smaller the $\theta_{P(x), Q(q)}$ is, the smaller the $QD(q, B(x))$ is likely to be. The proofs of Lemma 3 and Lemma 4 are introduced in the subsequent section.

4.3.3 Generating Probing Sequence. Unlike Multi-Probe that maintains a local probing sequence for each hash table, GMP generates a global probing sequence from all L hash tables. We illustrate the advantage of GMP over Multi-Probe as follows.

EXAMPLE 3. Fig. 3 shows two hash tables after the projection by using SRP. Assume that, for the first hash table, we have $a_{1,1} = (1.0, 0.1, -0.8)$, $a_{2,1} = (0.2, 0.9, -1.3)$. For the second hash table, we have $a_{1,2} = (0.6, -0.6, 1.1)$, and $a_{2,2} = (0.4, 0.9, -1.2)$. The projected values of q are $(\zeta_{1,1}(q), \zeta_{2,1}(q)) = (-1.6, 2.4)$ and $(\zeta_{1,2}(q), \zeta_{2,2}(q)) = (-3.0, 2.6)$. We denote $TiBj$ as the j -th bucket in the i -th hash table. The QD between q and each hash bucket is shown in Fig. 4(a).

By examining a same number of candidate points (excluding the repeated points), say 8 points, we compare the accuracy of GMP and Multi-Probe. In Multi-Probe, we examine 4 candidate points in each

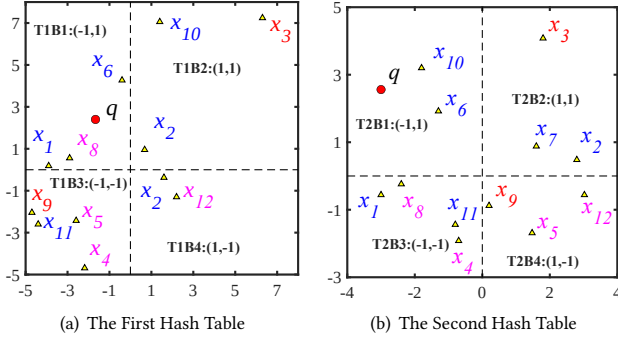


Figure 3: Point Projection on Two Hash Tables

Bucket	T1B1	T1B2	T1B3	T1B4	T2B1	T2B2	T2B3	T2B4
QD	0	2.7	5.8	8.5	0	9.0	6.6	15.6

(a) QDs of buckets

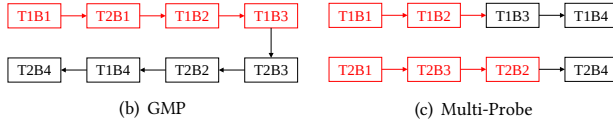


Figure 4: Probing Sequences of GMP and Multi-Probe

hash table based on the local probing sequence in 4(c). In GMP, we examine 8 candidate points based on the global probing sequence in 4(b). We ~~annotate~~ the examined buckets with red color. In GMP, we examine T1B1, T2B1, T1B2, and T1B3, and return x_9 . In Multi-Probe, we examine T1B1, T1B2, T2B1, T2B3, and T2B2, and return x_6 . This example reveals that GMP enables to achieve a higher accuracy than Multi-Probe by examining a same number of points.

However, it is expensive to compute the QDs of all $2^K \cdot L$ buckets to q and sort them to construct a complete global probing sequence. Fortunately, by maintaining a small-size heap, we enable to obtain the bucket with the least QD among all unseen buckets efficiently.

Let $\zeta_{i,j}(q)$ be the i -th entry of the projected vector in the j -th hash table. For a bucket $B = (b_1, \dots, b_K)$, from Eq. 15, we know that only the entries $\zeta_{i,j}(q)$ that have $h_{i,j}(q) \neq b_i$ contribute to the QD between q and B . Therefore, we can represent B by a set $S^j(B)$, which only records the indexes of the entries that differ from those of $h_{i,j}(q)$.

$$S^j(B) = \{i \mid b_i \neq h_{i,j}(q), 1 \leq i \leq K\}.$$

We reform Eq. 15 as follows.

$$\text{QD}(q, B) = \sum_{i \in S^j(B)} |\zeta_{i,j}(q)|^2, i \in S^j(B). \quad (17)$$

For simplicity, given the projected vector of q , we compute a sorted projected vector based on the value of $|\zeta_{i,j}(q)|$ so that we have $|\zeta_{i,j}(q)| < |\zeta_{i',j}(q)|$ for any $i < i'$ in the sorted projected vector. As a result, the smaller the entries in $S^j(B)$ are or the fewer entries the $S^j(B)$ has, the smaller the QD(q, B) is. Note that, by recording the indexes of the corresponding entries in the original projected vector of q , it is easy to reconstruct the hash bucket for a given set. Therefore, we proceed to generate a sequence of sets.

In the probing phase, we first probe the L buckets where q is located in the L hash tables. We maintain a min-heap H that stores the candidate sets in ascending order of QD. Initially, we insert the set $S^j(B) = \{1\}$ into H for each hash table, since the hash bucket B that corresponds to $S^j(B) = \{1\}$ has the smallest QD to q in the j -th hash table. Next, we continue to process the top set of H and insert two sets into H that are generated by two operations Shift() and Expand(), respectively.

Let $i_{\max}$ be the maximum entry of the current top set $S^j(B)$.

- (1) Shift($S^j(B)$) replaces $i_{\max}$ by $i_{\max} + 1$ in $S^j(B)$ and returns $S^j(B')$, which ensures that $S^j(B')$ is the set to generate that contains $S^j(B) \setminus \{i_{\max}\}$ and has the smallest QD.
- (2) Expand($S^j(B)$) adds the entry $i_{\max} + 1$ into $S^j(B)$ and returns $S^j(B')$, which ensures that $S^j(B')$ is the set to generate that contains $S^j(B)$ and has the smallest QD.

When $i_{\max} = K$, no new set will be inserted into H . The Shift() and Expand() operations ensure the correctness of the generated probing sequence.

4.3.4 Adaptive Early Termination (AET). After the transformation, a c -MIPS query is converted to a c' -ANNS query. An LSH method can not solve a c' -ANNS problem directly, but can solve a threshold-based (θ, c') -ANNS problem, which is defined as follows:

DEFINITION 7 ((θ, c')-ANNS). Given the query point q and a dataset $\mathcal{D}$, an angle threshold θ , and approximation $c' > 1$. The (θ, c') -ANNS query returns the following result:

- (1) If there exists at least one point x in $\mathcal{D}$ that $\theta_{q,x} < \theta$, it returns a point x' in $\mathcal{D}$ that $\theta_{q,x'} < c' \cdot \theta$;
- (2) If there does not exist a point x in $\mathcal{D}$ that $\theta_{q,x} < c' \cdot \theta$, it returns an empty set.

To conduct a c' -ANNS via LSH, we need to conduct a sequence of (θ, c') -ANNS queries by enlarging θ . In other words, we have to execute a sequence of (I, c) -MIPS queries by decreasing the inner product I . Similarly, the (I, c) -MIPS query is defined as follows.

DEFINITION 8 ((I, c)-MIPS). Given the query point q and a dataset $\mathcal{D}$, an inner product threshold I , and approximation $c < 1$. The (I, c) -MIPS query returns the following result:

- (1) If there exists at least one point x in $\mathcal{D}$ that $q^T x > I$, it returns a point x' in $\mathcal{D}$ that $q^T x' > c \cdot I$;
- (2) If there does not exist a point x in $\mathcal{D}$ that $q^T x > c \cdot I$, it returns an empty set.

It is easy to see that an (I, c) -MIPS query corresponds to a (θ, c') -ANNS query. So parameters θ and c' can be determined by a given I and c as follows.

$$\theta = \arccos\left(\frac{I}{M_i \cdot \|q\|}\right), \quad c' = \frac{\arccos\left(\frac{c \cdot I}{M_i \cdot \|q\|}\right)}{\arccos\left(\frac{I}{M_i \cdot \|q\|}\right)} \quad (18)$$

In the point probing process, we develop an adaptive early termination (AET) condition. For better understanding, we first present a normal termination (NT) condition as follows. Let T be the maximum number of points to probe, and λ be a QD threshold. For a (θ, c') -ANNS query, the process terminates when one of the following conditions satisfies:

- (1) We verify T candidate points;

(2) Let B^* be the bucket next to probe, we have $\text{QD}(q, B^*) > t$.

To ensure that NT returns correct (θ, c') -ANNS result, we determine the values of T and t as follows.

$$\begin{cases} (1 - \varphi(t; \theta))^L = 1/e \\ T = 2nL \cdot \varphi(t; c'\theta) \end{cases} \quad (19)$$

The intuition of Eq. 19 is that: when the first condition is satisfied, we have found a correct (θ, c') -ANNS result with at least probability $1 - 1/e$; when the second condition is satisfied, since there are at most $n \cdot \varphi(t; c'\theta)$ false positive points in a hash table, by using Markov's inequality, we have examined all false positive points in L hash tables with at least probability $1/2$.

$$1 - \frac{1 - (1 - \varphi(t; c'\theta))^L}{2L \cdot \varphi(t; c'\theta)} > 1 - \frac{L \cdot \varphi(t; c'\theta)}{2L \cdot \varphi(t; c'\theta)} = \frac{1}{2}.$$

NT is designed in a similar way of many mainstream LSH methods [9, 12, 35, 40], which guarantees to answer a c^2 -MIPS problem with a constant probability $1/2 - 1/e$ [12]. As mentioned, we answer a c^2 -MIPS query by conducting multiple (I, c) -MIPS queries. Let x^* be q 's exact MIPS point with $q^\top x^* = I_{\max}$, and we set $I = I_{\max}$ for the first (I, c) -MIPS query. For each (I, c) -MIPS query, we convert the (I, c) -MIPS query to a (θ, c') -ANNS query based on Eq. 18 and use NT to terminate the probing process. Once we find a point x with $q^\top x \geq cI$, we return it as the result. Otherwise, we decrease I to cI and conduct a (cI, c) -MIPS query.

NT employs two fixed parameters T and t for all queries. However, the number of points to probe to reach a given accuracy is different for query points, which restrains the performance when facing a data skewness issue. Motivated by this, we develop an adaptive early termination (AET) condition that "adaptively" takes both the best found result and $\text{QD}(q, B)$ into consideration. Assume that B^* is the next bucket to probe and I_0 is the current found maximum inner product.

We define two events according to I_0 and $\text{QD}(q, B)$.

- **E1**: For a point x with $q^\top x \geq I_0/c$, the hash bucket where x is located is in a certain hash table is still unseen.
- **E2**: For a point x with $q^\top x \geq I_0/c$, in L hash buckets where x is located, there is at least one that is still unseen.

When the probability that E2 happens, $\Pr[E_2]$, is small enough, the probability that we can find a better result in the remaining buckets is very small. So, it is reasonable to terminate the query process. Therefore, we define AET conditions as follows by setting a failure probability p_τ :

$$\begin{cases} \alpha_1 = \varphi(\text{QD}(q, B^*); \arccos(\frac{I_0}{c \cdot M_i \cdot \|q\|})) \\ \alpha_2 = 1 - (1 - \alpha_1)^L < p_\tau \end{cases} \quad (20)$$

Here, α_1 equals to $\Pr[E_1]$ and $\alpha_2 = 1 - (1 - \Pr[E_1])^L$ equals to $\Pr[E_2]$. Based on the intermediate query results, $\text{QD}(q, B^*)$ and I_0 , AET verifies whether $\Pr[E_2]$ is within the allowed failure probability p_τ . When AET is satisfied, it indicates that the found results in $\mathcal{D}_i$ is correct to be the results of c -MIPS. Compared to NT, the advantages of AET are:

- (1) The establishment of AET depends not on a fixed predefined threshold T and t but on the intermediate query results,

Algorithm 1: Global Multi-Probing (GMP)

Input: A query point q and partitions $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s, c, p_\tau, L$ hash tables

Output: A point

```

1 Compute hash values of  $q$ ;
2 Let  $x_0$  be the found best MIPS result;
3 for  $i = s \rightarrow 1$  do
4    $M_i \leftarrow \max_{x \in \mathcal{D}_i} \|x\|$ ;
5   if  $q^\top x_0 \geq cM_i$  then
6     Break;
7   do
8      $B^* \leftarrow$  the bucket with the smallest QD in the all
       unseen buckets;
9     Verify the points in  $B^*$  and update  $x_0$ ;
10  while AET is not satisfied;
11 return  $x_0$ ;
```

which reduces many unnecessary candidates especially when the data is skew;

- (2) AET is more efficient since we do not need to evaluate an initial $I_{\max}$ and conduct multiple (θ, c') -ANNS query;
- (3) AET tightens the approximation ratio from c^2 to c and directly solves a c -MIPS query rather than a c^2 -MIPS query, which incurs a smaller query cost.

We describe how to answer a c -MIPS problem by our solution that adopts AET as the termination condition in Algorithm 1.

THEOREM 1. Let $|\mathcal{D}_i| = n_i$. For each $\mathcal{D}_i$, by setting $p_\tau = \frac{\gamma}{n_i}$, where γ is a constant, Algorithm 1 answers a c -MIPS problem with a constant probability $e^{-\gamma}$.

PROOF. If there exists a point x that $q^\top x \geq I_0/c$, the probability that it is not found via GMP that adopts AET as the termination condition is at most p_τ . Assume there are totally N_i points whose inner product with q exceeds I_0/c . When AET satisfies, the probability that they are all found is $(1 - p_\tau)^{N_i} > (1 - p_\tau)^{n_i}$. Usually, n_i is large and thus $(1 - p_\tau)^{n_i} = \left(1 - \frac{\gamma}{n_i}\right)^{n_i} \approx e^{-\gamma}$. In this case, the result returned by GMP is correct c -MIPS. $\square$

In practice, $p_\tau = 0.1$ suffices to provide a good performance. GMP can be easily adapted to return k points with the maximum inner products. We call the c -MIPS with k results as (c, k) -MIPS. Specifically, to answer a (c, k) -MIPS query, we only need to set I_0 in Algorithm 1 to the current found k -th maximum inner product. We give an example to show how we answer a $(0.5, 2)$ -MIPS problem as follows.

EXAMPLE 4. In Fig. 1, we have $\mathcal{D}_1 = \{x_4, x_5, x_8, x_{12}\}$, $\mathcal{D}_2 = \{x_1, x_2, x_6, x_7, x_{10}, x_{11}\}$, and $\mathcal{D}_3 = \{x_3, x_9\}$. When a query q comes, we conduct query process in $\mathcal{D}_3$ first. We probe the hash tables according to the global probing sequence in Fig. 4(b) until the termination condition, AET, is satisfied. In this process, x_3 is found in T1B2 and x_9 is found in T1B3. Next, we continue conducting query in $\mathcal{D}_2$. AET is satisfied after probing T1B2 and $I_0 = q^\top x_6 = 16$. Since $I_0 > 0.5 \cdot M_1$, we terminate the query process and return x_9 and x_6 as results.

5 THEORETICAL ANALYSIS

We proceed to provide theoretical analysis for GMP. First, we present the theoretical guarantees of GMP. Second, we introduce the algorithm analysis.

5.1 Theoretical Guarantees of GMP

5.1.1 Property of Quantization Distance. We prove Lemmas 3 and 4 that reveal that the quantization distance is an effective indicator for probing.

For a query point q and a data point x whose angle with q is $\theta \in (0, \pi)$, since $\text{QD}(q, B(x))$ is the sum of a set of random variables, to determine the distribution of $\text{QD}(q, B(x))$, we first analyze the distribution of each random variable in $\text{QD}(q, B(x))$. Let $h_{i,j}$ be the i -th hash function in the j -th hash table. We define each item in $\text{QD}(q, B(x))$ as

$$\mu_{i,j}(q, x) = \begin{cases} |\zeta(q)|^2, & h_{i,j}(q) \neq h_{i,j}(x) \\ 0, & h_{i,j}(q) = h_{i,j}(x) \end{cases}$$

$\text{QD}(q, B(x))$ can be rewritten as $\sum_{i=1}^K \mu_{i,j}(q, x)$. Therefore, the distribution of $\text{QD}(q, B(x))$ can be determined by the distribution of $\mu_{i,j}(q, x)$.

LEMMA 5. Let ω be a non-negative number, the cumulative distribution function of $\mu_{i,j}(q, x)$ is

$$\psi(\omega; \theta) = \Pr[\mu_{i,j}(q, x) \leq \omega] = 1 - \frac{\theta}{\pi} + \int_0^{\sqrt{\omega}} 2\phi(-u \cdot \cot \theta) d\phi(u),$$

where $\phi(x)$ is the cumulative distribution function of the standard normal distribution.

It is easy to prove Lemma 5 according to the property of normal distribution. Then, the probability density function of $\mu_{i,j}(q, x)$ can be written as follow via $\psi(\omega; \theta)$,

$$g(\omega; \theta) = \frac{\partial \psi(\omega; \theta)}{\partial \omega} = \begin{cases} \frac{1}{\sqrt{\omega}} \phi(-\sqrt{\omega} \cdot \cot \theta) f(\sqrt{\omega}), & \omega > 0 \\ (1 - \frac{\theta}{\pi}) \delta(\omega), & \omega = 0 \\ 0, & \omega < 0 \end{cases}$$

Here $\delta(\cdot)$ is Dirac Delta function [14, 39] and $f(x)$ is the probability density function of the standard normal distribution.

We then prove Lemma 3.

PROOF. Given a point x , for any two different hash tables, let $B(x)$ be one of the buckets where x is located at in these two hash tables. Since $\zeta_{i,j}(q)$ ($j = 1, 2, i = 1 \dots K$) are independently and identically distributed, the distribution of $\text{QD}(q, B(x))$ is

$$\Pr[\text{QD}(q, B(x)) \leq \omega] = \Pr[\sum_{i=1}^K \mu_{i,j}(q, x) \leq \omega] = \int_0^{\omega} g^{K*}(u; \theta) du$$

where $g^{K*}(u; \theta)$ is the K -fold convolution of $g(u; \theta)$ [36] on u . It indicates that no matter which hash table $B(x)$ belongs to, $\text{QD}(q, B(x))$ follows the same distribution. $\square$

Next, we provide a proof for Lemma 4.

PROOF. Let $\varphi^{(n)}(\omega; \theta)$ be the cumulative distribution function of $\text{QD}(q, B(x))$ when $K = n$. We prove Lemma 3 by the mathematical induction. Let $P(n)$ be the statement that $\varphi^{(n)}(\omega; \theta)$ monotonically decreases with θ . We give a proof by induction on n .

Base case: Show that the statement holds for the smallest positive integer number $n = 1$. When $n = 1$, $\varphi^{(1)}(\omega; \theta) = \psi(\omega; \theta)$. Since

$$\frac{\partial \psi(\omega; \theta)}{\partial \theta} = -\frac{1}{\pi} + \frac{1}{\pi} [1 - \exp(-\frac{\omega}{2 \sin^2 \theta})] < 0$$

So, $\psi(\omega; \theta)$ monotonically decreases with θ , then $P(1)$ is clearly true.

Inductive step: Show that for any $k \geq 1$, if $P(k)$ holds, then $P(k+1)$ also holds. Since

$$\varphi^{(k+1)}(\omega; \theta_1) = \int_0^{\omega} g^{(k+1)*}(u; \theta) du$$

According to the definition of convolution, we have

$$\begin{aligned} \int_0^{\omega} g^{(k+1)*}(u; \theta) du &= \int_0^{\omega} g(u; \theta_1) * g^{k*}(u; \theta_1) du \\ &= \int_0^{\omega} \int_0^u g(t; \theta_1) g^{k*}(u-t; \theta_1) dt du \end{aligned}$$

Interchanging the order of integral, we have

$$\begin{aligned} &\int_0^{\omega} \int_0^u g(t; \theta_1) g^{k*}(u-t; \theta_1) dt du \\ &= \int_0^{\omega} g(t; \theta_1) dt \int_t^{\omega} g^{k*}(u-t; \theta_1) du \\ &= \int_0^{\omega} g(t; \theta_1) dt \cdot \varphi^{(k)}(\omega-t; \theta_1) \\ &> \int_0^{\omega} g(t; \theta_1) \varphi^{(k)}(\omega-t; \theta_2) dt = \int_0^{\omega} g(u; \theta_1) * g^{k*}(u; \theta_2) du \end{aligned}$$

Similarly, we have

$$\begin{aligned} &\int_0^{\omega} g(u; \theta_1) * g^{k*}(u; \theta_2) du \\ &> \int_0^{\omega} g(u; \theta_2) * g^{k*}(u; \theta_2) du = \varphi^{(k+1)}(\omega; \theta_2) \end{aligned}$$

That is, the statement $P(k+1)$ also holds true, establishing the inductive step.

Conclusion: Since both the base case and the inductive step have been proved as true, by mathematical induction the statement $P(n)$ holds for every positive integer number n . $\square$

5.2 Algorithm Analysis

We consider N_0 as a constant according to [17]. Let $n_i = |\mathcal{D}_i|$, so $n_i = O(1)$. Therefore, we enable to set L and K as constants to compute MIPS in $\mathcal{D}_i$. The space cost of GMP for $\mathcal{D}_i$ is $O(n_i)$. Assume the probability that a false positive point is verified is β , the expected number of candidates for $\mathcal{D}_i$ is at most $\beta n_i + 1$, β is much smaller than 1.

THEOREM 2. GMP has space cost $O(n)$ and time cost $O(\beta n d)$, where β is much smaller than 1.

PROOF. Since L and K are constants, the space consumption is $O(n)$. The query time cost comes from two parts: 1) generating probing sequence; and 2) verifying the inner products between candidate points and q . In each partition $\mathcal{D}_i$, the former has cost $O(n_i \log n_i)$ and the latter has cost $O(\beta n_i d)$. Note that, the probing sequence is the same in all partitions since we use the same

Table 3: Datasets

Dataset	$n (\times 10^3)$	d	Dataset	$n (\times 10^3)$	d
Netflix	17	300	Trevi	100	4,096
Audio	54	192	NUS	269	500
MovieLens	60	300	YahooMusic	625	300
MNIST	60	784	GIST	1,000	960
Cifar	60	1,024	TinyImages5M	5,000	384

$K \cdot L$ hash functions to build the index. So, we only have to generate the probing sequence once. Therefore, the total query time is $O(n_i \log n_i + \sum_{i=1}^s \beta n_i d) = O(\beta n d)$. $\square$

6 EXPERIMENT

We report on extensive experiments with real-world datasets that offer insight into the performance of GMP. In particular, we aim to answer the following questions:

- Q1: Why is RXT a better transformation method than XT in GMP?
- Q2: How does GMP benefit from the adaptive early termination (AET) strategy?
- Q3: How does one set the parameters that affect the performance of GMP?
- Q4: How does GMP perform relative to the competitors under default settings?
- Q5: How does GMP compare with the competitors on the Recall-Time dimensions and the Ratio-Time dimensions?

6.1 Experimental Settings

All the algorithms are implemented in C++ and are compiled with the O3 optimization. All experiments are run on a Windows machine with an AMD 2.9GHz CPU and 32GB memory.

6.1.1 Datasets and Query Sets. We use 10 well-adopted real datasets as shown in Table 3. Specifically, *Netflix*, *MovieLens*, and *YahooMusic* are often used in recommendation systems [5, 22]. The other datasets are used widely in similarity search [24].

For queries, we randomly select 200 points from each dataset and repeat each experiment 20 times. We vary the value of k in $\{1, 10, 20, \dots, 100\}$ and set the default value to 50. We vary the value of c in $\{0.1, 0.2, \dots, 0.9\}$ and set the default value to 0.8.

6.1.2 Competing Algorithms. We compare GMP with 7 competitors:

- (1) **L2-ALSH** [32] uses the L2-Transformation and the (K, L) -bucketing strategy.
- (2) **Sign-ALSH** [33] uses the Correlation-Transformation and the (K, L) -bucketing strategy.
- (3) **Simple-LSH** [28] uses the XBOX-Transformation and the (K, L) -bucketing strategy.
- (4) **Range-LSH** [38] applies the norm-ranging and then uses the Simple-LSH.
- (5) **H2-ALSH** [17] uses the XBOX-Transformation and the QALSH [16] after applying the norm-ranging.
- (6) **RPT** [21] uses the XBOX-Transformation and the randomized partition trees.

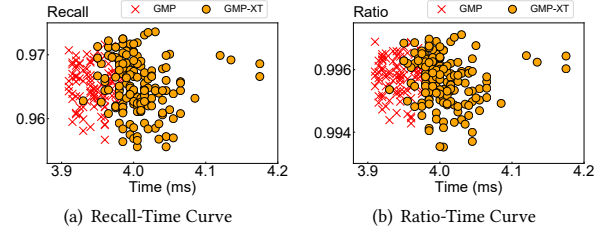


Figure 5: Comparison on RXT and XT

(7) **LScan** verifies all points and return the best k points.

6.1.3 Parameter Settings. GMP needs to consider the parameters b_0 , N_0 , L , K , and p_τ . We set $K = 12$, $N_0 = 2^K = 4096$ and $p_\tau = 0.1$ as default. b_0 and L are determined according to the data distribution. Typical values are $b_0 = \sqrt{0.95}$ and $L = 5$. For H2-ALSH, we set $c_0 = 2.0$ and $N_0 = 5000$ [17]. For the other algorithms, we perform tuning to obtain parameter settings that achieve the best performance.

6.1.4 Evaluation Metrics. We use three performance metrics: query time (ms), overall ratio, and recall, where the query time evaluates efficiency and the overall ratio and recall evaluate result quality. For a query q , we denote the result of a (c, k) -MIPS query by $R = \{x_1, x_2, \dots, x_k\}$. Let $R^* = \{x_1^*, x_2^*, \dots, x_k^*\}$ be the exact k MIPS points. The overall ratio and recall are computed as follows.

$$\text{OverallRatio} = \frac{1}{k} \sum_{i=1}^k \frac{q^\top x_i}{q^\top x_i^*} \quad (21)$$

$$\text{Recall} = \frac{|R \cap R^*|}{|R^*|} \quad (22)$$

6.2 Performance Evaluation

To evaluate the performance of GMP, we first demonstrate that RXT and AET are better choices for the transformation method and termination condition than are XT and NT. Then we conduct an evaluation to determine parameter settings for GMP. Next, we compare the performance of all algorithms with default settings on all datasets. Finally, we study the overall ratio-time and recall-time curves for all algorithms.

6.2.1 Comparison of RXT and XT (Q1). To compare RXT and XT, we denote by GMP-XT our solution with XT as the transformation method. For brevity, we only show the results on *YahooMusic*. We repeat the experiment for 100 times and plot findings in Fig. 5. We observe that GMP is more stable than GMP-XT in terms of both recall-time and ratio-time. In other words, GMP achieves better accuracy in less time. Two observations explain this. (1) In each experiment, the data distribution after applying RXT is more even. (2) For the 100 experiments, GMP generates a similar data partitionings. That is, the index structure changes only little across the experiments.

6.2.2 Comparison of AET and NT (Q2). To compare the performance of AET and NT, we denote by GMP-NT our solution with NT as the termination condition. We plot the recall-time and

Table 4: Performance Overview

		GMP	H2-ALSH	Simple-LSH	NR-ALSH	Sign-ALSH	RPT	L2-ALSH	LScan
Netflix	Query Time (ms)	1.41	3.90	20.4	2.38	19.6	19.4	20.9	51.3
	Overall Ratio	1.000	1.000	0.568	0.972	0.571	0.763	0.853	/
	Recall	1.000	1.000	0.3582	0.8721	0.3573	0.5688	0.7170	/
MovieLens	Query Time (ms)	1.96	7.66	17.1	2.10	13.8	17.5	16.2	21.6
	Overall Ratio	1.000	1.000	0.9618	0.9846	0.9349	0.9953	0.9020	/
	Recall	1.000	1.000	0.8917	0.9485	0.8278	0.9865	0.7953	/
YahooMusic	Query Time (ms)	6.07	10.4	18.7	16.3	21.8	12.2	68.7	323
	Overall Ratio	0.9998	0.9995	0.2254	0.9765	0.2357	0.1066	0.4972	/
	Recall	0.9972	0.9913	0.1013	0.9269	0.1344	0.1023	0.2942	/
Audio	Query Time (ms)	8.86	12.2	9.55	9.69	9.15	9.64	8.95	17.3
	Overall Ratio	0.9895	0.9796	0.9763	0.9878	0.9864	0.9792	0.9356	/
	Recall	0.8889	0.8177	0.8355	0.8841	0.8805	0.8617	0.6494	/
MNIST	Query Time (ms)	6.22	7.17	17.5	13.0	8.28	16.0	12.2	52.0
	Overall Ratio	0.9982	0.9975	0.9895	0.9951	0.9680	0.9846	0.9574	/
	Recall	0.9234	0.9086	0.7986	0.8921	0.5649	0.7411	0.4716	/
Cifar	Query Time (ms)	0.255	0.510	3.07	1.91	11.9	5.28	17.9	60.9
	Overall Ratio	1.000	0.9991	0.9899	0.9978	0.9674	0.9958	0.9848	/
	Recall	0.9986	0.9054	0.7692	0.9455	0.6402	0.8279	0.7774	/
Trevi	Query Time (ms)	1.27	6.63	10.4	8.40	77.9	17.75	34.29	332
	Overall Ratio	1.000	1.000	0.9977	1.000	0.9868	0.9955	0.9664	/
	Recall	1.000	1.000	0.7064	0.9992	0.7518	0.8572	0.5945	/
NUS	Query Time (ms)	3.16	3.21	31.4	23.1	29.0	47.9	034.9	180
	Overall Ratio	0.9986	0.9942	0.8391	0.9104	0.7040	0.7964	0.7567	/
	Recall	0.9726	0.9277	0.2605	0.6758	0.2390	0.4404	0.2679	/
GIST	Query Time (ms)	0.895	8.34	32.8	3.98	8.99	29.6	50.5	1046
	Overall Ratio	1.000	1.000	0.9024	0.9697	0.9168	0.946	0.8488	/
	Recall	1.000	1.000	0.3694	0.6745	0.4322	0.5556	0.2112	/
TinyImages5M	Query Time (ms)	0.614	15.7	41.7	19.1	123	78.6	104	3115
	Overall Ratio	0.9990	0.9883	0.9606	0.9918	0.9704	0.9301	0.9700	/
	Recall	0.9014	0.6134	0.2601	0.6991	0.4032	0.1596	0.4321	/

ratio-time curves when varying p_r for GMP and varying T and t for GMP-NT—see Fig. 6. We observe that GMP takes less time than GMP-NT to reach a same recall or overall ratio. A key reason is that AET adaptively decides when to terminate so that fewer candidates are verified. Normally, to achieve a certain accuracy, the required number of candidate points to examine differs for each query point. NT does not consider this issue.

6.2.3 Comparison of GMP and Multi-Probe. We compare the performance of GMP and Multi-Probe with the same number of candidate points. By setting the number of candidate points T as a fixed value, GMP examines T points according to the global probing sequence; Multi-Probe examines T/L points in each hash table according to each local probing sequence. We plot the recall-time and ratio-time curves when varying T —see Fig. 7. We observe that GMP takes less time than Multi-Probe to reach a same recall or overall ratio. It indicates that with the same number of candidate points, the results returned by GMP is more accurate than that by Multi-Probe.

6.2.4 Parameter Study on GMP (Q3). We study the effect of L on GMP. Fig. 8 shows that when L increases, we obtain a larger overall ratio and a larger recall that takes less query time. Thus, a large L is always a good choice for query efficiency, since a large L leads to an accurate distance estimation. In addition, a large L makes AET easier to satisfy so that fewer candidates need to be verified. However, a larger L inevitably incurs a larger space consumption. We observe that when L reaches 5, the query time gain gradually levels out. To balance space consumption and query efficiency, we set $L = 5$ as the default value.

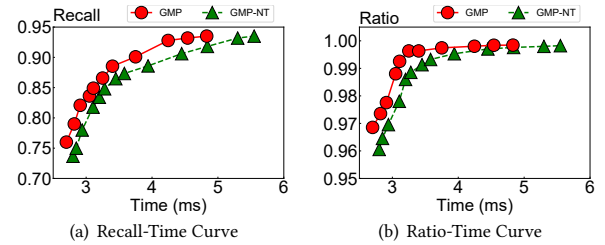


Figure 6: Comparison on AET and NT

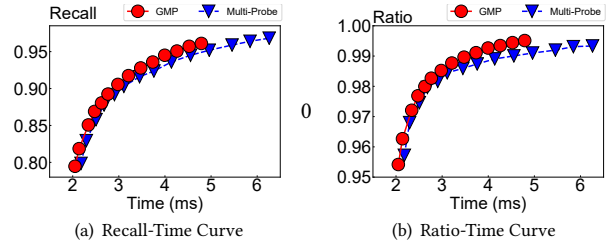


Figure 7: Comparison on GMP and Multi-Probe

6.2.5 Performance Overview (Q4). We report the query time (ms), overall ratio, and recall of all algorithms with default settings on all datasets in Table 4. GMP has better query time than all competitors on all datasets, and the overall ratio and recall are also

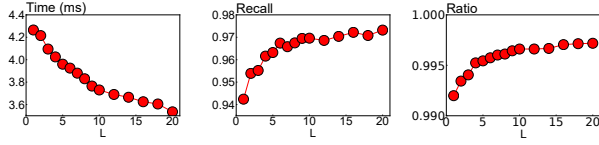


Figure 8: Performance of GMP when Varying L

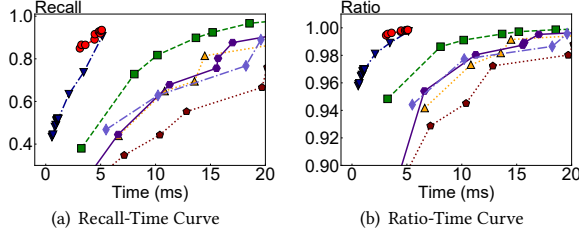


Figure 9: Recall/Ratio-Time Curve on MNIST

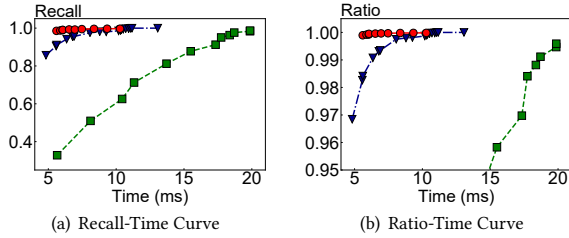


Figure 10: Recall/Ratio-Time Curve on YahooMusic

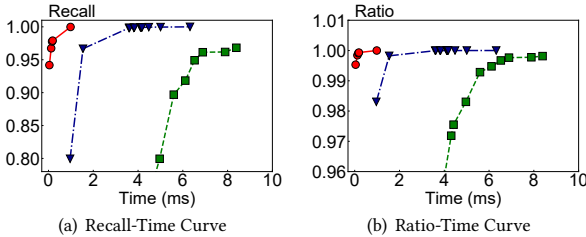


Figure 11: Recall/Ratio-Time Curve on GIST

better than those of the competitors. We observe that *Cifar* and *GIST* incur the smallest query time. These datasets have nearly the same dimensionality but different cardinality, which indicates that the query time is not heavily affected by the dataset cardinality. This can be explained by the norm-ranging that to some extent eliminates the effect of cardinality. Another interesting observation is that the algorithms without norm-ranging, i.e., Simple-LSH, Sign-ALSH, RPT, and L2-ALSH, perform much worse than the algorithms with norm-ranging, i.e., GMP, H2-ALSH, and Range-LSH. This reveals that the norm-ranging plays an important role in MIPS.

6.2.6 Effect of k (Q4). We study the performance when varying the value of k in $\{1, 10, 20, \dots, 100\}$. For brevity, we only report the performance on datasets *MNIST*, *YahooMusic*, and *GIST*—see Figs. 12–14. GMP achieves the best performance on all metrics. H2-ALSH is always second-best. As k increases, all algorithms achieve smaller overall ratio and recall. The query time of the algorithms without norm-ranging is relatively steady. This is because the number of candidates verified is nearly unaffected by k . However, the query time of the algorithms with the norm-ranging, i.e., GMP, H2-ALSH, and Range-LSH, increases with k . In addition, the termination conditions are affected by the k -th result, so the number of candidates verified increases with k .

6.2.7 Recall-Time and OverallRatio-Time Curves (Q5). We plot the recall-time and overall ratio-time curves for all algorithms on three datasets when varying c , L , and K —Fig. 9–11. All algorithms return more accurate results with larger query times. GMP again performs the best on all metrics, and only H2-ALSH and Range-LSH have comparable performance. On large datasets, the query time of the other algorithms increase rapidly with the cardinality of dataset. This is due to the norm-ranging. We observe that the cardinality of *GIST* is 20 times of that of *MNIST*, while *GIST* incurs less query time than *MNIST*. This is because the points in *GIST* are distributed evenly so that it suffices to find good results in enough points with large norms. Among the three best algorithms, GMP performs better than H2-ALSH and Range-LSH. First, GMP-LSH has a more accurate distance estimator than H2-ALSH and Range-LSH. Second, GMP has a better transformation method and termination condition, which prevents it from examining unnecessary candidates when retrieved results are good enough.

7 RELATED WORK

The MIPS problem is a classic problem in the database community. Early methods employ space partitioning trees, such as the M-tree [22], and the cone-tree [30]. However, these methods are computationally expensive in high-dimensional spaces due to the curse of dimensionality. Therefore, approximate solutions to the MIPS problem are studied [4, 21, 32]. LSH is a mainstream approximation method for solving the MIPS problem. Since inner product is not a metric, an asymmetric transformation method is applied to convert the MIPS problem into NNS. Shrivastava et al. [32] propose an L2-Transformation and a Correlation-Transformation [33]. XBOX-Transformation [4] eliminates the distortion errors and is adopted by Simple-LSH [28]. Yan et al. [38] improve Simple-LSH with a norm-ranging strategy. H2-ALSH [17] improves the performance by using norm-ranging and QALSH [16]. In addition, learning-based methods are proposed for MIPS [8, 11, 31]. Shen et al. [31] propose a binary code learning framework that preserves the inner product among raw data vectors. Next, neighbor graph based methods are proposed for MIPS [25, 27, 34] that were originally used to solve approximate NNS problems.

8 CONCLUSION

We propose GMP for maximum inner product search with theoretical guarantees. First, we develop a novel transformation called RXT that converts this problem into approximate nearest neighbor search. Second, we propose a global multi-probing strategy as well

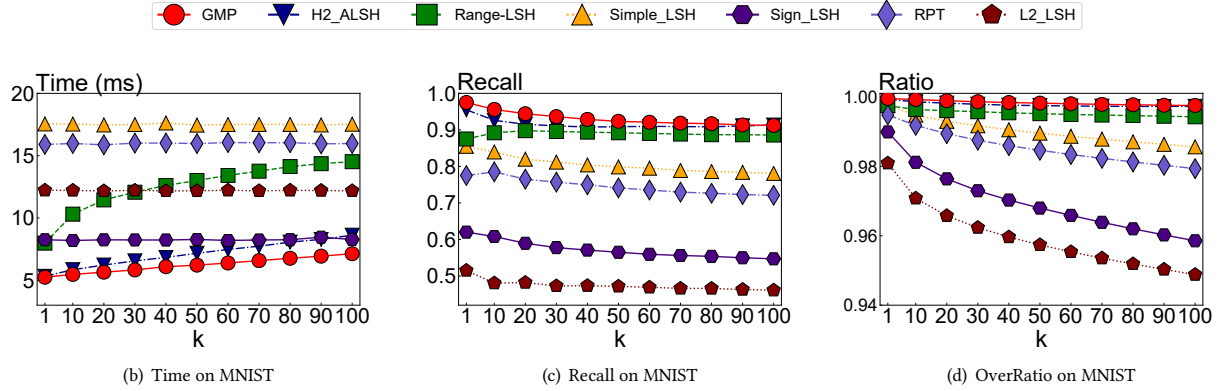


Figure 12: Performance on MNIST when Varying k

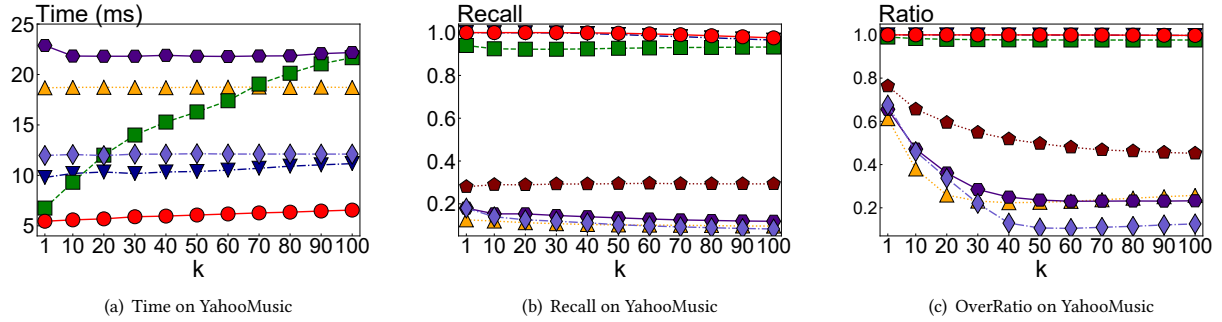


Figure 13: Performance on YahooMusic when Varying k

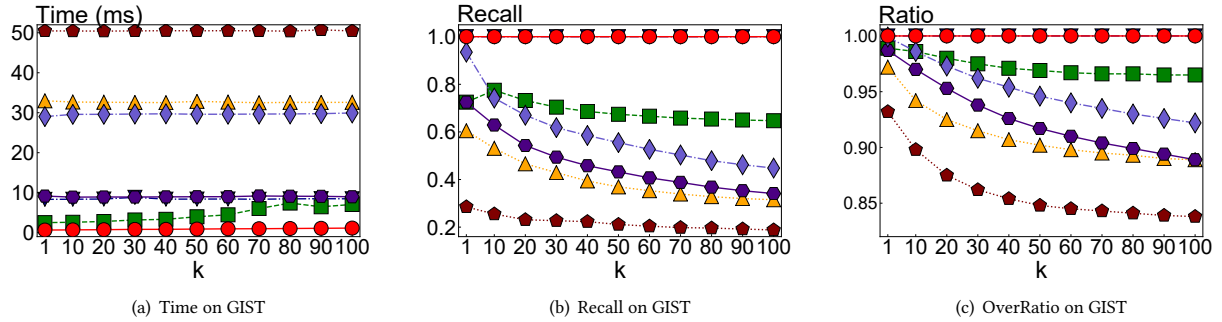


Figure 14: Performance on GIST when Varying k

as an early termination condition to examine high quality candidates. An experimental study shows that GMP outperforms all its competitors in terms of both efficiency and accuracy. Specifically, GMP improves the query time by an average of 35% when compared to the closest competitor (H2-ALSH). When all competitors take approximately the same query time, GMP improves recall by about 20–40% over the closest competitor (H2-ALSH).

REFERENCES

- [1] Mohamed Hussein Abdi, George Onyango Okeyo, and Ronald Waweru Mwangi. 2018. Matrix Factorization Techniques for Context-Aware Collaborative Filtering Recommender Systems: A Survey. *Sci* 11, 2 (2018), 1–10.
- [2] Thomas Dybdahl Ahle, Rasmus Pagh, Ilya P. Razenshteyn, and Francesco Silvestri. 2016. On the Complexity of Inner Product Similarity Join. In *PODS*. 151–164.
- [3] Alexandr Andoni, Piotr Indyk, Huy L. Nguyen, and Ilya P. Razenshteyn. 2014. Beyond Locality-Sensitive Hashing. In *SODA*. 1018–1028.
- [4] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nave, and Ulrich Paquet. 2014. Speeding up the Xbox recommender system using a Euclidean transformation for inner-product spaces. In

- RecSys. 257–264.
- [5] Robert M. Bell and Yehuda Koren. 2007. Lessons from the Netflix prize challenge. *SIGKDD Explorations* 9, 2 (2007), 75–79.
 - [6] Moses Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. 380–388.
 - [7] Lijie Chen. 2018. On The Hardness of Approximate and Exact (Bichromatic) Maximum Inner Product. *Electronic Colloquium on Computational Complexity (ECCC)* 25 (2018), 26.
 - [8] Xinyan Dai, Xiao Yan, Kelvin Kai Wing Ng, Jiu Liu, and James Cheng. 2020. Norm-Explicit Quantization: Improving Vector Quantization for Maximum Inner Product Search. In *AAAI*. 51–58.
 - [9] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SoCG*. 253–262.
 - [10] Thomas L. Dean, Mark A. Ruzon, Mark Segal, Jonathon Shlens, Sudheendra Vijayanarasimhan, and Jay Yagnik. 2013. Fast, Accurate Detection of 100, 000 Object Classes on a Single Machine. In *CVPR*. 1814–1821.
 - [11] Marco Fraccaro, Ulrich Paquet, and Ole Winther. 2016. Indexable Probabilistic Matrix Factorization for Maximum Inner Product Search. In *AAAI*. 1554–1560.
 - [12] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Vldb*. 518–529.
 - [13] Sarel Har-Peled, Piotr Indyk, and Rajeev Motwani. 2012. Approximate Nearest Neighbor: Towards Removing the Curse of Dimensionality. *Theory Comput.* 8, 1 (2012), 321–350.
 - [14] Sadri Hassani. 2009. Dirac delta function. In *Mathematical methods*. Springer, 139–170.
 - [15] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *WWW*. 173–182.
 - [16] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search. *PVLDB* 9, 1 (2015), 1–12.
 - [17] Qiang Huang, Guihong Ma, Jianlin Feng, Qiong Fang, and Anthony K. H. Tung. 2018. Accurate and Fast Asymmetric Locality-Sensitive Hashing Scheme for Maximum Inner Product Search. In *KDD*. 1561–1570.
 - [18] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.
 - [19] Prateek Jain and Ashish Kapoor. 2009. Active learning for large multi-class problems. In *CVPR*. 762–769.
 - [20] Thorsten Joachims, Thomas Finley, and Chun-Nam John Yu. 2009. Cutting-plane training of structural SVMs. *Mach. Learn.* 77, 1 (2009), 27–59.
 - [21] Omid Keivani, Kaushik Sinha, and Parikshit Ram. 2018. Improved maximum inner product search with better theoretical guarantee using randomized partition trees. *Machine Learning* 107, 6 (2018), 1069–1094.
 - [22] Noam Koenigstein, Parikshit Ram, and Yuval Shavitt. 2012. Efficient retrieval of recommendations in a matrix factorization framework. In *CIKM*. 535–544.
 - [23] Naama Kraus, David Carmel, Idit Keidar, and Meni Orenbach. 2016. NearBucket-LSH: Efficient Similarity Search in P2P Networks. In *SISAP (Lecture Notes in Computer Science, Vol. 9939)*. 236–249.
 - [24] Jinfeng Li, Xiao Yan, Jian Zhang, An Xu, James Cheng, Jie Liu, Kelvin Kai Wing Ng, and Ti-chung Cheng. 2018. A General and Efficient Querying Method for Learning to Hash. In *SIGMOD*. 1333–1347.
 - [25] Jie Liu, Xiao Yan, Xinyan Dai, Zhirong Li, James Cheng, and Ming-Chang Yang. 2020. Understanding and Improving Proximity Graph Based Maximum Inner Product Search. In *AAAI*. 139–146.
 - [26] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search. In *Vldb*. 950–961.
 - [27] Stanislav Morozov and Artem Babenko. 2018. Non-metric Similarity Graphs for Maximum Inner Product Search. In *NeurIPS*. 4726–4735.
 - [28] Behnam Neyshabur and Nathan Srebro. 2015. On Symmetric and Asymmetric LSHs for Inner Product Search. In *ICML*, Vol. 37. 1926–1934.
 - [29] Jennifer Nguyen and Mu Zhu. 2013. Content-boosted matrix factorization techniques for recommender systems. *Stat. Analysis and Data Mining* 6, 4 (2013), 286–301.
 - [30] Parikshit Ram and Alexander G. Gray. 2012. Maximum inner-product search using cone trees. In *KDD*. 931–939.
 - [31] Fumin Shen, Wei Liu, Shaoting Zhang, Yang Yang, and Heng Tao Shen. 2015. Learning Binary Codes for Maximum Inner Product Search. In *ICCV*. 4148–4156.
 - [32] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). In *NIPS*. 2321–2329.
 - [33] Anshumali Shrivastava and Ping Li. 2015. Improved Asymmetric Locality Sensitive Hashing (ALSH) for Maximum Inner Product Search (MIPS). In *UAI*. 812–821.
 - [34] Shulong Tan, Zhixin Zhou, Zhaozhuo Xu, and Ping Li. 2019. On Efficient Retrieval of Top Similarity Vectors. In *EMNLP/IJCNLP (1)*. 5235–5245.
 - [35] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2009. Quality and efficiency in high dimensional nearest neighbor search. In *SIGMOD Conference*. 563–576.
 - [36] Christoph Haehling von Lanzau and William N Lundberg. 1974. The n-fold convolution of a mixed density and mass function. *ASTIN Bulletin: The Journal of the IAA* 8, 1 (1974), 91–103.
 - [37] Mengshuang Wang, Jun Ma, Shanshan Huang, and Peizhe Cheng. 2015. Combining Positive and Negative Feedbacks with Factored Similarity Matrix for Recommender Systems. In *WAIM*, Vol. 9098. 233–246.
 - [38] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-Ranging LSH for Maximum Inner Product Search. In *NeurIPS*. 2956–2965.
 - [39] Lin Zhang. 2020. Dirac delta function of matrix argument. *International Journal of Theoretical Physics* (2020), 1–28.
 - [40] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *PVLDB* 13, 5 (2020), 643–655.
 - [41] Yuxin Zheng, Qi Guo, Anthony K. H. Tung, and Sai Wu. 2016. LazyLSH: Approximate Nearest Neighbor Search for Multiple Distance Functions with a Single Index. In *SIGMOD*. 2023–2037.